



# **CSE 4513**

## **Lec – 12**

# **Open-Source Compliance/ Release & maintenance**

*All contents are based on textbooks, research articles, online resources, and personal experience.*





# OPEN-SOURCE SOFTWARE (OSS) LICENSE



## ➤ What is it?

- ✓ OSS license is a type of license for computer software and other products that allows **the source code, blueprint or design to be used, modified and/or shared** under **defined terms and conditions**.
- ✓ OSS license is a legal and binding contract between the author and the user, declaring that the software can be used in commercial applications under certain conditions.
- ✓ In general, OSS licenses makes the source code available under terms that allow for modification and redistribution without having to pay the original author.

# IMPACT OF FOSS LICENSES

---



FOSS licenses may impact:

- Use of the software
- Modification of the software
- Maintenance of the software
- Distribution of the software and its derivatives
- Intellectual property rights (IPR)

# PERMISSIONS GRANTED

---



FOSS licenses may permit:

- Modification of the source code
- Recompilation of the software
- Redistribution of the original source code, modified source code and/or binaries
- Integration of the software with proprietary software

# OSI APPROVED OSS LICENSES



One popular set of open-source software licenses are those approved by the Open-Source Initiative (OSI) based on their Open-Source Definition(OSD).

The following OSI-approved licenses are popular, widely used, or have strong communities:

- ✓ Apache License 2.0
- ✓ BSD 3-Clause "New" or "Revised" license
- ✓ BSD 2-Clause "Simplified" or "FreeBSD" license
- ✓ GNU General Public License (GPL)
- ✓ GNU Library or "Lesser" General Public License (LGPL)
- ✓ MIT license
- ✓ Mozilla Public License 2.0
- ✓ Common Development and Distribution License
- ✓ Eclipse Public License version 2.0

# OPEN-SOURCE LICENSES : **PERMISSIVE**



In many cases, many open-source developers felt the best way to make software “free”, is to make the licenses the least restrictive as possible.

- ✓ We call these licenses “**permissive**” licenses.
- ✓ Permissive licenses have very little restrictions and obligations, letting you effectively do whatever you want at no cost (e.g., copy it, distribute it, sell it, build your own products on top of it, etc.)

E.g., **MIT License:**

*“Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions...”*

# OPEN-SOURCE LICENSES : COPYLEFT (PROTECTIVE)



In other cases, some open-source developers felt the best way to make software “free” is to include a “viral” element to ensure software always remains “free” and infects other software to make other software “free” too.

- ✓ We call these licenses “**viral**” or “**copyleft**” licenses.
  - (1) If you distribute the “copyleft” software, often you must disclose source code.
  - (2) If you build software that is based on the “**copyleft**” software, often you must also disclose your proprietary source code.

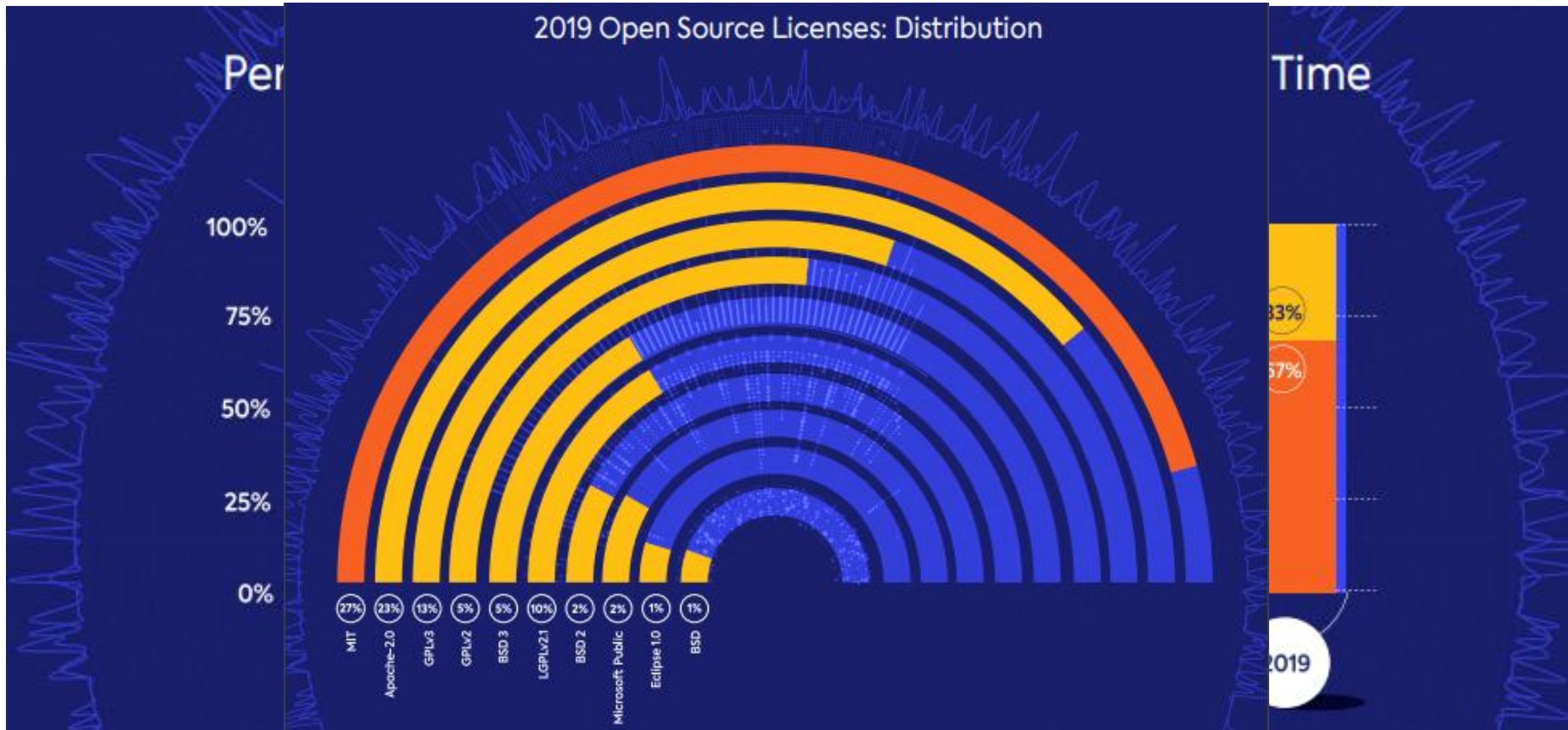
E.g., General Public License (GPL), Affero General Public License (AGPL)



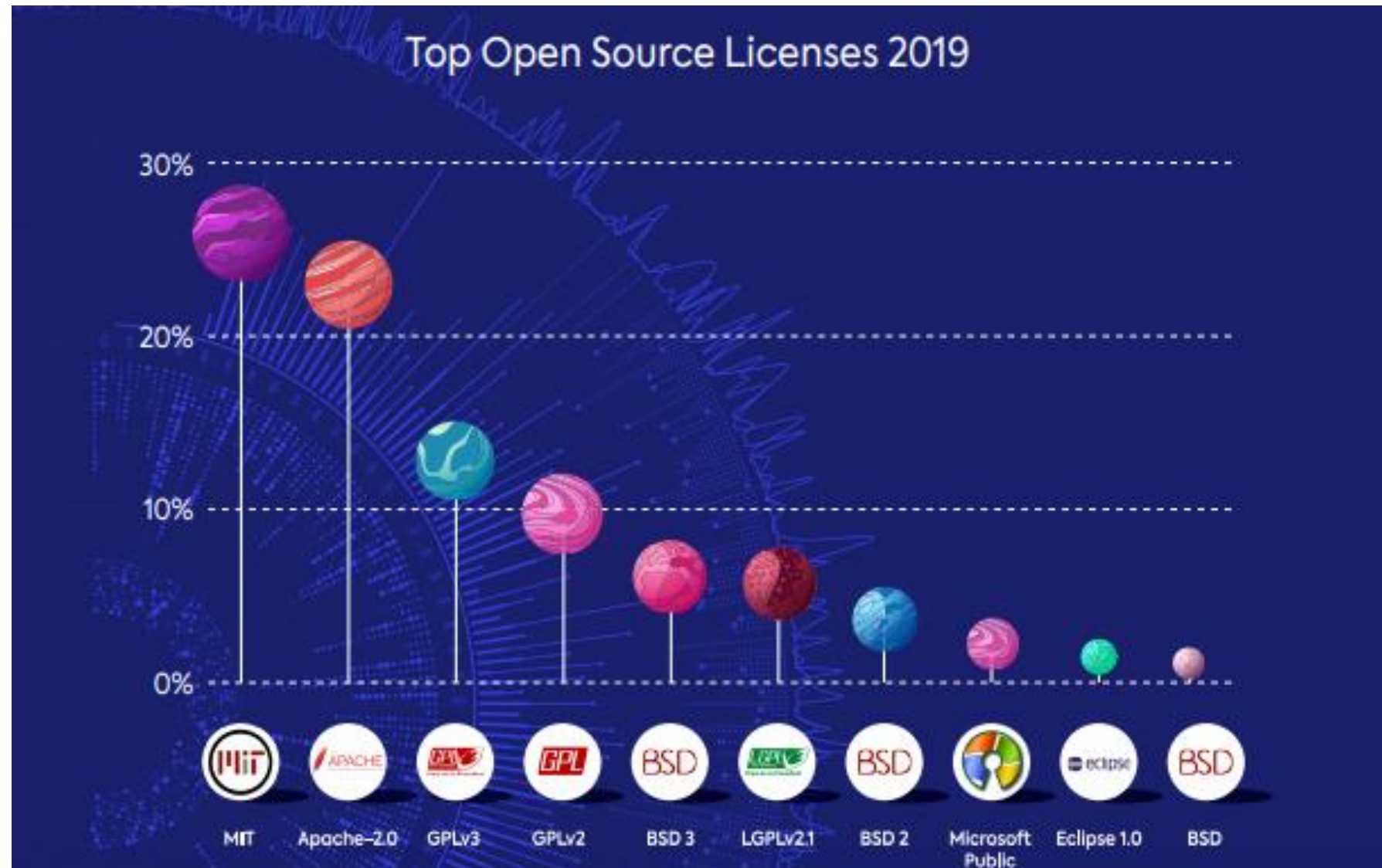
# THE TREND



## PERMISSIVE OPEN-SOURCE LICENSES CONTINUE TO TREND



## STILL AT THE TOP:THE MIT OPEN-SOURCE LICENSE



# BEERWARE LICENSE.



Many licenses are written by programmers with a sense of humor.



```
/*
 * -----
 * "THE BEER-WARE LICENSE" (Revision 42):
 * <phk@FreeBSD.ORG> wrote this file.  As long as you retain this notice you
 * can do whatever you want with this stuff.  If we meet some day, and you think
 * this stuff is worth it, you can buy me a beer in return.  Poul-Henning Kamp
 * -----
 */
```

# THE FACT



- ✓ There are over 25 million repositories on GitHub, over 430,000 projects on SourceForge.
- ✓ These days, a developer may do a Google search, find five open-source products that fit his[/her] need and quickly integrate one of them into a product.
- ✓ Many software developers work under the following false misconception:
  - “OSS is freely available, so I can use it without any kind of restriction”.
- ✓ This is completely wrong: many OSS, even when freely available, are in fact governed by their own licensing conditions, which imply, in most cases and depending on the use of the OSS, strict contractual license restrictions.

# MITIGATING RISKS THROUGH COMPLIANCE PRACTICES

---



- Identification of the origin and license of used software.
- Identification of license obligations.
- Fulfillment of license obligations when product ships.



# WHAT IS FOSS COMPLIANCE

---



- ✓ Compliance means that users of FOSS must observe all the copyright notices and satisfy all the license obligations for the FOSS they use.
- ✓ In addition, companies using FOSS in commercial products, while complying with the terms of FOSS licenses, want to protect their intellectual property and that of third-party suppliers from unintended disclosure.

# WHAT IS FOSS COMPLIANCE



Open-Source Compliance refers to the aggregate of

- policies
- processes
- training
- tools

that enables an organization to effectively use open-source software and contribute to open communities while

- respecting copyrights,
- complying with license obligations, and
- protecting the organization's intellectual property and that of its customers and suppliers.

# COMPLIANCE OBJECTIVES

---



- ✓ Comply with third party software supplier contractual obligations in light of FOSS licensing obligations
- ✓ Facilitate effective usage of FOSS in commercial products
- ✓ Protect commercial product differentiation while complying with FOSS contractual obligations



# COMPLIANCE BENEFITS

---



- ✓ Increased understanding of the benefits of FOSS and how it impacts your organization.
- ✓ Increased understanding of the costs and risks associated with using FOSS.
- ✓ Better relations with the FOSS community and FOSS organizations.
- ✓ Increased knowledge of available FOSS solutions.
- ✓ Be prepared for possible acquisition, sale, new product or service release, where compliance assurance is mandatory before the completion of any of these transaction.
- ✓ Improve your overall FOSS strategy using the results from your compliance program.

# SW RELEASE MANAGEMENT..

# WHAT IS A SW RELEASE

---



- ✓ A SW release is the launch of a new SW or a combination of features that will provide value to customers or users.
- ✓ For your customers, it's a promise of new value they can look forward to embedding in their everyday work and activities.
- ✓ For your internal teams, a release helps them plan their work and when they'll be needed to launch great products.

A release is not just the act of providing access to new technical functionality. Instead, think of a **release as the date** when the company is **ready to deliver a new customer experience** and support every customer interaction point associated with it.

# WHAT IS RELEASE MANAGEMENT?



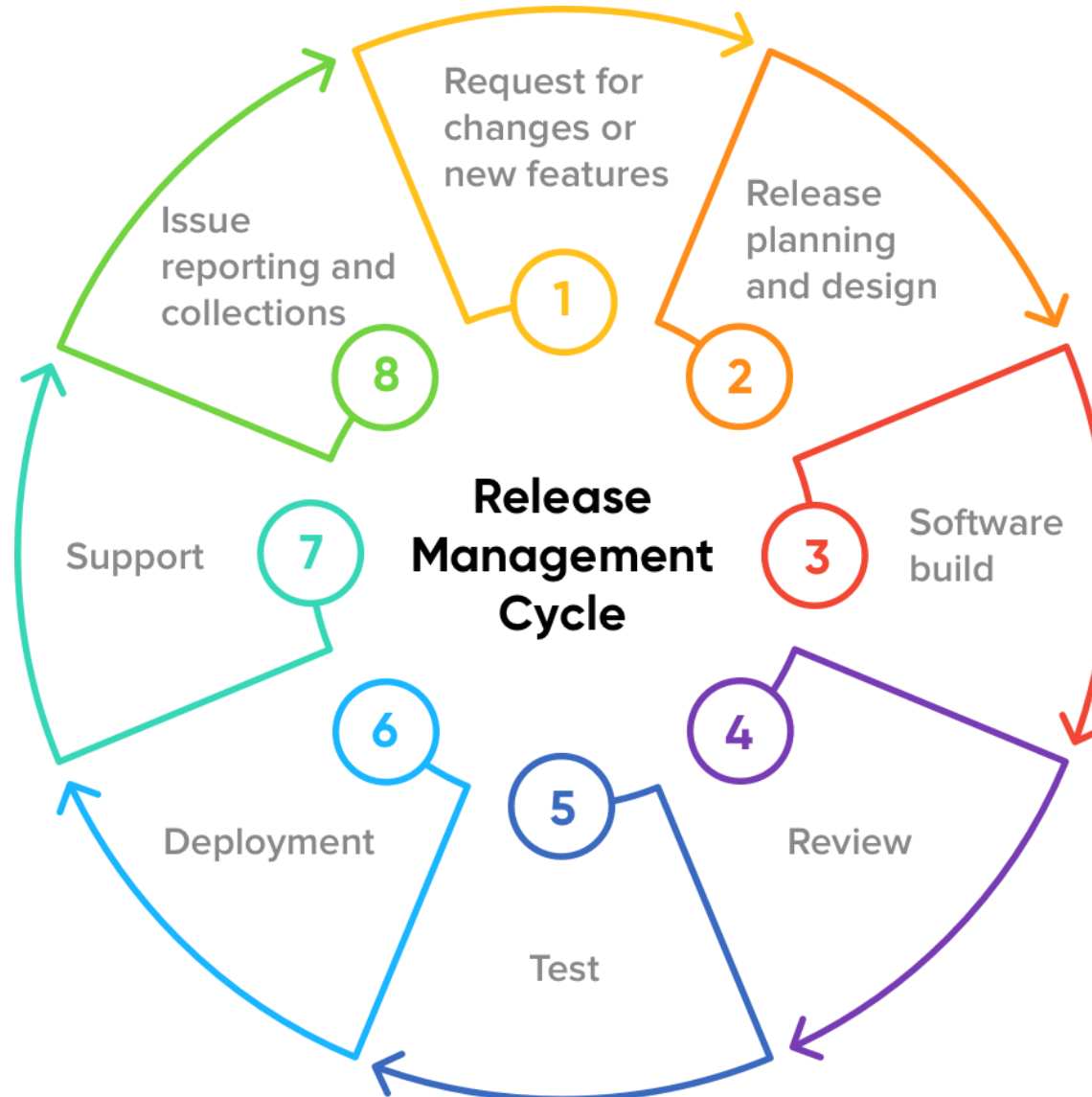
- ✓ Release Management is the **process** of planning, building, testing and deploying hardware and software and the version control and storage of software.
- ✓ Its purpose is to ensure that a consistent method of deployment is followed.
- ✓ It reduces the likelihood of incidents as a result of rollouts and ensures that only tested and accepted versions of hardware and software are installed at any time.

# THE OBJECTIVES RELEASE MANAGEMENT



- ✓ Deploy application changes into production without disrupting the business.
- ✓ To be classified as successful, the release must meet the following objectives:
  - (a) Deployed on time
  - (b) Deployed on budget
  - (c) Have no or negligible unwanted impact on existing customers
  - (d) Satisfy the requirements of new customers, competitive pressure and/ or technological advances.

# RELEASE MANAGEMENT CYCLE



# THE PROCESS – BUILD RELEASE



- ✓ With the release plan finalized, you can start designing and building the product for release.
- ✓ This is the actual “development” of the product based on the requirements outlined in the release plan.
- ✓ Once all the issues that may have come up are addressed, it’s time to subject the build to real-world scenario testing.
- ✓ This could take several iterations and allows the team to identify any bugs or issues that may arise in a real-world environment.
- ✓ As issues are identified, the build is sent back for development at stage two. In other words, within the iterative release management process, the work may flow from stage two to stage three and back again until the release is approved.



# THE PROCESS – PREPARE RELEASE



- ✓ This step is to put the finishing touches on the product, taking into account everything that was learned in UAT.
- ✓ Release preparation also includes a final quality review by the QA team.
- ✓ During the review, the QA team will conduct final checks to ensure the build meets the minimum acceptable standards and business requirements outlined in the release plan.
- ✓ Once the review is completed, the functional team will validate the findings and finalize the release for deployment.
- ✓ Before the build can deploy into a live environment, it must be approved by the product owner.

# THE PROCESS – DEPLOY RELEASE



- ✓ It's time to release your product into the wilds of the live production environment.
- ✓ Besides simply sending the build out into production, the deployment stage also includes messaging and education on the product to both the end user and your company at large.
- ✓ For instance, users should be notified of changes with the release and how to operate within the new features. Depending on how significant the changes were, you may need to provide robust and ongoing training to get everyone up to speed.
- ✓ Finally, during the deployment stage, the development team should meet to assess the release's performance and discuss how the deployment went.
- ✓ If there are any lingering issues, those should be identified and documented for the team to address in the next iteration..

# RELEASE MANAGEMENT IN AGILE+DEVOPS



- ✓ In Agile methodologies, agile release trains align to value streams to deploy releases.
- ✓ Release units are delivered every 2-week sprint and release packages are deployed to production every 10-12 weeks.
- ✓ As an enterprise adopts DevOps, the release manager also coordinates with environment managers for testing and deployment planning for release packages
- ✓ Release management is responsible for coordinating with the DevOps manager to monitor the continuous integration and delivery in the DevOps pipeline.

# WHAT DOES A RELEASE MANAGER DO?

---



- Understands the business needs and their priorities, and under what circumstances those priorities can change.
- Works with business leaders, product owners, IT project teams, and operations staff to ensure every release contains the correct features.
- Changes the release scope or re-prioritize release features according to information from project and portfolio managers.
- Has a clear picture of development dependencies, and how changes to one part of a product can affect the stability of the whole.
- Schedules release unit dependencies into release packages.

# WHAT DOES A RELEASE MANAGER DO?

---



- Understands the bandwidth and work capacity of each team involved in development.
- Understands the availability of resources and environments for testing.
- Schedules builds and testing according to team and resource bandwidth and availability.
- Create release plans, including governance and approval requirements.
- Ensure compliance of new releases with governance requirements.
- Optimizes value creation at every step, from feature check-in to deployment.
- Schedules seamless release deployments.

# RELEASE MANAGER



| Operations Manager            | Product Manager           | Release Manager      | DevOps Manager             | CIO                    | Breadth / General Knowledge |
|-------------------------------|---------------------------|----------------------|----------------------------|------------------------|-----------------------------|
| Develop policies & procedures | Understand business needs | Provide reporting    | Improve collaboration      | Navigate tech trends   |                             |
| Monitor IT systems            | Define scope of project   | Adjust release scope | Testing automation         | Set strategy           |                             |
| Direct IT technicians         | Make activity schedules   | Create release plans | Implement CI/CD            | Customer engagement    |                             |
| Resolve help desk escalations | Risk & time management    | Ensure compliance    | Find & address bottlenecks | Establish partnerships |                             |
| Depth / Specialized Knowledge |                           |                      |                            |                        |                             |

# RELEASE MANAGEMENT BEST PRACTICES



Some natural best practices:

- **Automate** as much as possible, both on the devops and testing side. Automation cuts down on the human cost, allowing more rapid iteration, and reduces the chance for false positives and negatives (computers tend to make less mistakes than humans).
- **Have clear requirements**, and from these make testable acceptance criteria. There should be no ambiguity about whether the software is ready to ship.
- Minimize user impact of a release by **minimizing or eliminating downtime** and testing for regressions before release.
- Make things **immutable wherever possible**. Instead of modifying the configuration of an existing machine, deploy a complete image that contains all of the configuration. This avoids bugs from appearing due to an unexpected series of actions.

# RELEASE NOTE



- ✓ Release notes is a document, which is released as part of the final build that contains new enhancements that went in as part of that release and also the known issues of that build.

## Release Notes Format:

- ✓ **Header** - Name of the document, which carries product name, release number, release date, release note date and version.
- ✓ **Overview** - An overview of the product and changes to the recent software version.
- ✓ **Purpose** - An overview of the purpose of the release notes which lists the new feature, enhancements and defects of the current build.
- ✓ **Issue Summary** - Provides description about the defect.
- ✓ **End-User Impact** - Provides information about the end-users impact due to the defect.
- ✓ **Contact** - Support contact information.



# RELEASE VERSIONING



- ✓ Versions and Releases are denoted using a quadruplet of integers to indicate Major, Minor, and Revision (Rev); Build numbers are for internal tracking and verification of the software build process and will not be visible to customers are part of the software version number.

**<MAJOR>. <MINOR>. <REVISION>.<BUILD>**

Example: “**2.4.7.1333**”

The version above is reference as version 2.4.7, Where: **Major** release is 2, **Minor** release is 4, **Rev release** is 7 and **Build** is not used except for internal tracking/deployments

# MAJOR RELEASE



- ✓ A Major Release is a full product release of the software.
- ✓ It generally contains new customer-facing functionality and represents a **significant change to the code base** comprising the software product or family of products, or is used to represent a significant marketing change or direction in the product.

## Scope:

- ✓ Any change to the code base that prevents backwards compatibility (e.g. Addition or Removal of features, changes in the DB schema or API commands).
- ✓ Any new functionality that is customer facing.
- ✓ Any large marketing push that accompanies the product and redirects the product.

## Frequency:

Market driven

## Audience:

New customers.

Existing customers with qualifying contracts

Existing customers desiring to upgrade to the new feature set

# MINOR RELEASE



- ✓ A **Minor Release** of the software may be comprised a rollup of several branched releases, enhancements/extensions to existing features or interfaces driven by internal or external requirements.
- ✓ external requirements could be driven by enhancements to meet new sales area, internal requirements could be enhancements aligned to a new marketing push.

## Scope:

- ✓ Minor enhancements and features that do not affect compatibility with its associated major or current minor releases. New features or functionality that does minimally effect the interfaces
- ✓ Addition of new features or functions to meet a new sales area that doesn't affect existing customers of the release. To accompany or communicate a new marketing initiative.
- ✓ Error corrections and maintenance.
- ✓ A rollup of branched releases.

## Frequency:

Enhancement or Market driven

## Audience:

New customers.

Existing customers with qualifying contracts

Existing customers desiring to upgrade to the new feature set

# REVISION RELEASE



- ✓ A Revision (Rev) is a build of all or part of the software that is initially distributed to an internal audience, specifically software quality assurance, for software validation.
- ✓ If the Rev is successfully validated and accepted, this version is “released” to manufacturing. If defects are found that prevent the Rev from successfully being validated and prevent the release to manufacturing, the Rev value will be incremented prior to the next validation cycle.

## **Scope:** •

- ✓ Releasing build to SQA for validation

## **Frequency:**

- ✓ As necessary, but typically every 1 to 3 months.

## **Audience:**

- ✓ Existing customers with qualifying contracts.

# BUILD RELEASE



- ✓ A **Build Release** is a build of all or part of the software distributed to an internal audience, these releases should have targeted feature enhancements and issue resolution documented to allow testing in the targeted/specific areas where the changes were implemented.

## Scope:

- ✓ Resolves a particular defect, typically of a critical Severity level with no viable workaround.

## Frequency:

- ✓ Extensively used during internal development.

## Audience:

- ✓ Internal developers
- ✓ Internal verification